Mancalagent

Mahmut Zahid Goksu * Ömer İslam † Emre Duzakin ‡

April 15, 2026

1 Problem definition

Mangala is a traditional Turkish strategy game that dates back centuries and is part of the larger family of Mancala-style games played around the world. It is a two-player game involving a playing board with small pits and stones, where players take turns distributing the stones strategically to capture their opponent's pieces and control the board. Known for its simple rules yet deep tactical possibilities, Mangala has been a popular pastime in Turkish culture in the Ottoman era, often associated with intelligence and strategic thinking. Mangala consists of 12 small pits, 6 for each player and 2 treasuries as illustrated in Fig.1. Each player aims to capture as many stones as possible by capturing stones to their own treasuries.

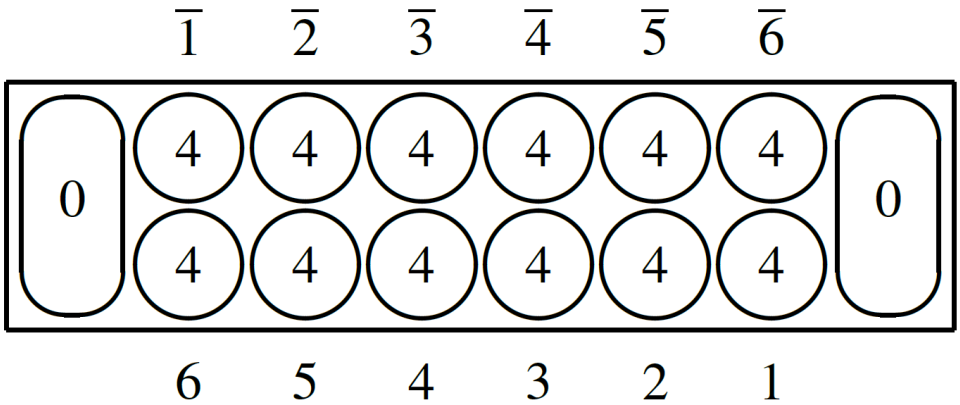

Figure 1: Illustration of a Mangala board. Each player has six small pits, each initially containing four stones. Additionally, each player has a treasury (also known as a store) located to their right, where captured stones are collected.

Kalah a similar game to our Mangala has been shown to be solved in "Solving Kalah" by Irving et al.[3]. Kalah has been shown to have a state-space complexity of 1.31×10^{13}, which can be considered a relatively large search space. [3] has shown that when played optimally, the first player can win by at least 3 stones. In this project, we aim to train a Reinforcement Learning (RL) agent using two main algorithms—Monte Carlo Tree Search and TD-Gammon—with the goal of emulating optimal play that can consistently win against average players.

*NO: 79520 Email: mgoksu21@ku.edu.tr
†NO: 79909 Email: oislam21@ku.edu.tr
‡NO: 69663 Email: eduzakin18@ku.edu.tr

2 Related Work

2.1 Game playing with Reinforcement learning

Reinforcement Learning has been applied to many games and games have been a popular domain for RL. AlphaZero has utilized Monte-Carlo tree search (MCTS) for state-value assignment has proven to be better than previous attempts, which have used alpha-beta search with domain-specific enhancements [5]. Additionally, AlphaZero has shown great promise against state-of-art Algorithms in various games, such as chess, Go, and shogi, proving the MCTS method to be able to perform regardless of the domain [5].

2.2 Mangala and Monte Carlo Tree Search

Mancala, which is the name of the larger family of games that includes Mangala, has been explored with RL in the past. Previous attempts at solving Mancala has primarily focused on the use of brute-force methods such as using mini–max game-tree-search algorithms.[4]However, such methods have the problem of assuming optimal play from a human which not the case most of time, resulting in suboptimal choices in practice[4]. Additionally, computational complexity of the search algorithms makes the real-time practice harder, due to the long decision making times[4]. As for The reinforcement learning solutions, the RL agents shown a higher win percentage when going second was around 60% as opposed to the claimed 70% win rate. Additionally, the agents has shown significant improvement in their performance against human players compared to the baseline, with only 150,500 games of training[1]. Recently, Monte Carlo Tree Search (MCTS) has also been explored as a more flexible search method for Mangala, showing promise in handling complex branching factors at the same time relying less on exhaustive searches[2].

2.3 TD-Gammon Approach

TD-Gammon is a reinforcement learning algorithm that applies temporal-difference learning. It was originally developed for Backgammon and taught itself by playing repeatedly against its own internal policy[7]. We will adapt TD-Gammon for Mangala by allowing the agent to learn incrementally from each move, updating its policy based on differences between predicted outcomes and actual results. This iterative process helps the agent refine its strategy without exhaustive search, making it a good candidate alongside MCTS.

3 The current RL formulation of the problem

- **State:** Due to the fully observable nature of the game, our states representation is an array of the board along with the action taken within the state. In our implementation search board state is represented as a vector see in 1, where the first 5 indexes represent current players pits and P1 represents the store.

$$[1, 2, 3, 4, 5, 6, P1, 1', 2', 3', 4', 5', 6', P2]^T \tag{1}$$

- **Action:** As illustrated in Fig.1 the action space of an agent is any one of the pits on a players' respective side marked from 1-6.

- **Rewards:** Due to the nature of the game our rewards are sparse and are received in Win, lose and draw conditions for MCST algorithm, where as for the TD-Gammon rewards are received at each player move. Additionally there are several different dynamics that Mangala offers such as capturing stones, which can happen in several conditions. Such actions that benefit the user is also incorporated in the reward.

4 Baselines

Since Mangala has not been extensively explored in the context of reinforcement learning (RL), there are no published RL agents or established baselines for comparison. As a result we have conducted our own experiments which is mentioned in the results.

5 Implementation

We have used two different solutions to Mangala. The first of these is the MCTS algorithm. The second is the TD-Gammon algorithm

5.1 RL with Monte Carlo Tree Search

Monte Carlo Tree Search is an iterative algorithm that searches the state space and builds statistical evidence on the decisions available in particular states [6]. MCTS have been utilized in game playing in the past with AlphaGO [5], where they use a MCTS algorithm with pre-trained Fast-fast roll-out and supervised value network. Due to limited resources and data we were not able to train a fast-rollout and supervised value network but we have gotten the inspiration from AlphaGO. MCST consists of four step: Selection, Expansion, Simulation, and Backpropagation. In our implementation:

5.1.1 Selection

For our selection strategy,, we directly use the strategy used in AlphaGO. For selection we use the formula 4, where $Q(s_t, a)$ is given by 3 and $U(s_t, a)$ is given by 2. $P(s, a)$ represents the probability of an state and action. $P(s, t)$ is initially uniform for all possible actions and after the first iteration of our tree will be calculated using the Policy network REINFORCE and $N(s_t, a)$ represents the visit count of the edge $e(s, a)$

$$U(s, a) = c_{\text{puct}}\, P(s, a)\, \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \tag{2}$$

$$Q(s, a) = \frac{W(s, a)}{N(s, a)} \tag{3}$$

$$a_t = \arg\max_a \big(Q(s_t, a) + U(s_t, a)\big) \tag{4}$$

5.1.2 Expansion

For our expansions strategy we use a every search, every child expansion strategy. From this we mean that with each search, when encountered a leaf node of the built tree we expand each possible child of that leaf node. This allows us to build the tree faster due to the limited amount of moves possible (maximum possible action is 6) while also accumulating visits and values of each edge.

5.1.3 Simulation

For our simulation strategy we use a random roll-out due to the absence of labeled data to train a fast roll-out network.

5.1.4 Backpropagation

After receiving rewards from our simulations, we propagate back until the root node, incrementing visit count and values of edges in the path from the root to the expanded leaf node.

5.1.5 Training Policy and Value Network

The MCTS algorithm enforces a policy π_{mcst} with each iteration, and the resulting transitions can be used to train our policy and Value networks. We create two different networks NN_{policy} and NN_{value}. we use a simple linear layer followed by nonlinear transformation. In addition to training these networks we later want to use these to rebuild the MCTS tree to get even better result in using the NN_{policy} network for prior probabilities for edges and NN_{value} network for evaluating return given by formula 5 where z_t is the return from random roll-out.

$$V(s_t, a) = \lambda NN_{value}(s_t) - (1 - \lambda)z_t \tag{5}$$

In addition to this we have experimented with Both REINFORCE training and soft actor-critic like training. Soft actor-critic training uses independent positions from each game samples randomly according to batch size while REINFORCE uses a trajectory based sampling. In our REINFOCE implementation we have used trajectory's generated by our MCST as independent samples. to achieve this we have utilized a padding to create each batch and samples from these trajectories independently. Our results are shown in the result section of our report.

5.2 TD-Gammon algorithm

TD-Gammon is an derivation of the TD-Gamma algorithm, which is an online learning algorithm that updates the state values based on eligibility traces. TD-Gammon builds on top of the regular TD-Gamma idea by incorporating a deep network to approximate the value of the state rather than keeping track of the states in a tabular manner[8].
In our implementation on top of the regular TD-Gammon we have utilized a min-max agent in order to have better decision making. Mangala contains action which can result in an extra turn, so having an insight of the future promotes better moves and setups. Furthermore, we have also utilized replay buffers in order to have better decision making in the early game. As for our network, we have used a smaller network composed of four linear layer with Tanh activation between each layer.

6 Completed Work

We worked on this project in two stages: *implementation* and *evaluation*.

- **Implementation**
  - We built a complete Python environment for *Mangala* with an automated test suite.
  - We developed two learning pipelines:
    - *Monte-Carlo Tree Search (MCTS)* powered by lightweight policy and value networks trained via PyTorch self-play (25 k playouts per turn).
    - *TD-Gammon–style agent* featuring a four-layer network, depth-5 mini-max look-ahead, and a replay buffer for stable early learning.
  - We also refactored code for clarity, added command-line flags for hyper-parameters, and documented the training workflow.
- **Evaluation**
  - We ran 1 000-game benchmarks against a random baseline.
  - TD-Gammon agent wins 74–77% when moving first and with no observable advantage moving second.
  - Although Current MCTS prototype has not yet surpassed random play, we can observe signs of strategic understanding in its moves.

7 Evaluation Details, Results and Analysis/Discussion

7.1 Evaluation

For the evaluation of the agents, we have compared our agents against a random agent over thousand games. Furthermore, we evaluated the decision and strategy of the agents by observing its choices by playing against the agent ourselves. Finally, since in Mangala first player has an advantage we have also evaluated our agent going second against the random agent.

7.2 Results

7.2.1 TD-GAMMON Results

Our agent is able to achieve 74–77% win rate against the random agent when going first. This is an improvement over the random-agent vs random-agent win rate which is only around 51–53%. While the min-max agent was not able to increase the win percentage, it has shown to be better at recognizing strategic moves such as prioritizing the move which results in an extra turn (See fig.2 and fig.4).

As for going second, our agents is worse than the random-agent, around the 22-25%. To mitigate this issue we have used replay buffer to generalize to going second as well. As a result our going second win rate has increased to 41-43%. But this improvement came at the cost of going first win rate which has fallen down to 53%.

Additionally, we have tested the effect of epsilon over the performance of the agent. While there is no significant improvement in win rate gradually decreasing the epsilon over time allows for agent to recognize advantageous moves easier, as it was able to explore more. This can also be seen in the error graph where the decreasing epsilon performs worse in the middle of the training but is better at the end. Whereas the error of sharp cut of epsilon (0.9 for 0.2 of the episodes 0.1 for rest) has increasing error in the end of the training.

7.2.2 MCTS with REINFORCE

As stated in we have both experimented with on-policy (Trajectories path perserved) and off-policy (Independent play positions). We have found that in both methods due to our training scheme we have not been able to achieve any significant improvements in results in terms of gameplay. We have found that for each training the policy network had learned to pick the exact same pit for each state given to it. When we investigated the outputs logits of the model only the pit index 4 had gotten a positive logit while other pits had negative scores which made the model pick the index 4.

```
Player 0's turn
-------------------------------------------
       5'    4'    3'    2'    1'    0'
    +-----+-----+-----+-----+-----+-----+
    |  0  |  1  |  1  |  1  |  1  |  1  |
  4 +-----+-----+-----+-----+-----+-----+  1
    |  0  |  0  |  0  |  1  |  0  |  1  |
    +-----+-----+-----+-----+-----+-----+
       0     1     2     3     4     5
                  Player 0
-------------------------------------------
Action probabilities: [0.5, 0.5]
Selected action: 3
Player 1's turn
```

Figure 2: Action Probabilities for base TD

```
Player 0's turn
-------------------------------------------
       5'    4'    3'    2'    1'    0'
    +-----+-----+-----+-----+-----+-----+
    |  0  |  1  |  1  |  1  |  1  |  1  |
  4 +-----+-----+-----+-----+-----+-----+  1
    |  0  |  0  |  0  |  1  |  0  |  1  |
    +-----+-----+-----+-----+-----+-----+
       0     1     2     3     4     5
                  Player 0
-------------------------------------------
Action probabilities: [0.1112193614244461, 0.8887806534767151]
Selected action: 5
Player 0 gets an extra turn!
Player 0's turn
```

Figure 3: Replay action choice for TD with min-max

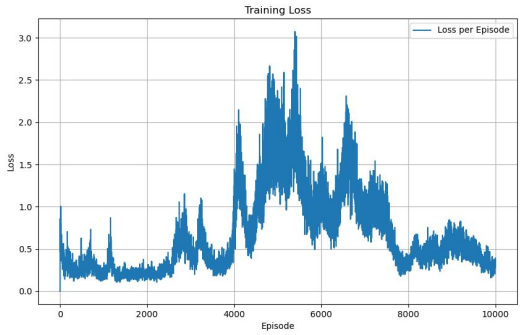

Figure 4: TD-Error for min-max with depth 5 and decreasing epsilon

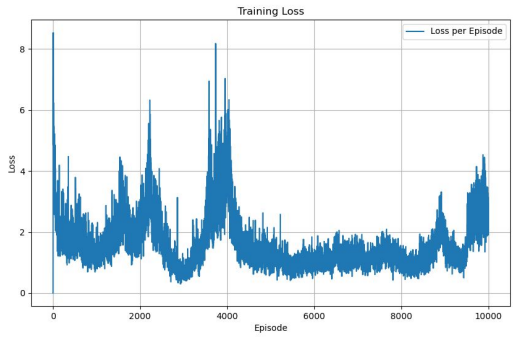

Figure 5: TD-Error for min-max with sharp epsilon decrease

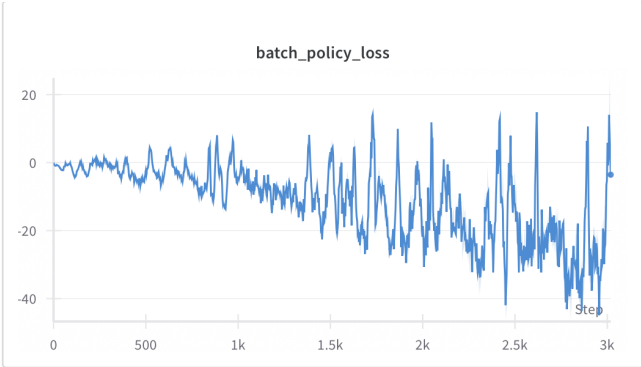

Figure 6: Policy loss on 25K MCTS runs, 3 EPOCH. All trajectories were perserved to have on-policy training

This is mostly due to the training process and the models architecture where we used the tanh activation as our nonlinearities. We have also considered the possibility that this was caused by the samples generated by the MCTS but no evidence was found that there was a bug on our MCTS implementation. The training plots show that overall off-policy training (fig.8, fig.9) was far more noisy and could not optimize leading to fluctuation policy and value losses, while on-policy (fig.7, fig.6) shows an improvement in Policy loss and value loss.

7.3 Discussion

We believe that our agent is able to learn the game and see some strategic more there is still room to be improvement. Additionally, our results have shown that our agent is tend to learn strategies for going first and our going second win rate is inversely proportional to the going first win rate. Our training is not limited to a single agents perspective and learns from both agents moves. However, splitting the networks for going first and going second might lead to better win rates and strategies.

8 Conclusion

Our TD-Gammon agent is successful at beating the random agent when going first but lacks the generalization to perform going second. We have utilized min-max and replay buffer to solve such issues and improve performance. While the methods improved recognizing strategic moves win rate of the agents does not increase significantly. Training separate agents for solely going first or second could have yieled better results.

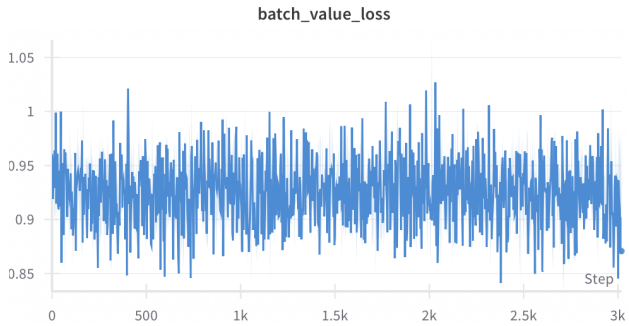

Figure 7: Value loss on 25K MCTS runs, 3 EPOCH. All trajectories were perserved to have on-policy training

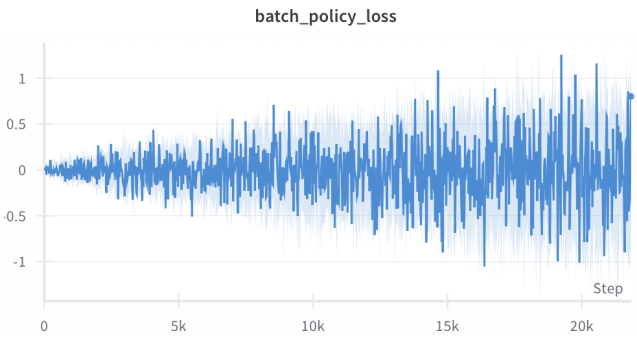

Figure 8: Policy loss on 25K MCTS runs, 3 EPOCH off-policy training

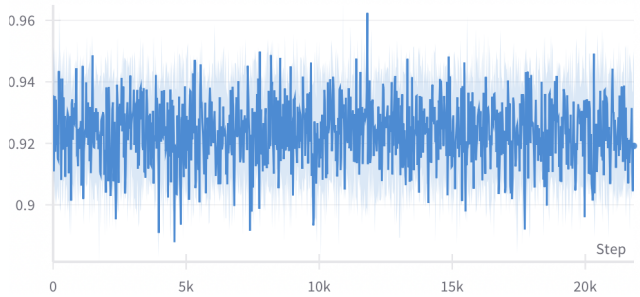

Figure 9: Value loss on 25K MCTS runs, 3 EPOCH off-policy training

References

[1] Tiffanie Birrell. Padawan to jedi: Using reinforcement learning to train an agent to play mancala. Honors College Thesis, 2019. Accessed: [date].

[2] Trevon J. Hunter. The exploration and analysis of mancala from an ai perspective. *Honors Theses*, (257):1–34, 04 2021.

[3] Geoffrey Irving, Jeroen Donkers, and Jos Uiterwijk. Solving kalah. *ICGA journal*, 23, 05 2003.

[4] Libor Pekař, Radek Matušů, Jiří Andrla, and Martina Litschmannova. Review of kalah game research and the proposition of a novel heuristic–deterministic algorithm compared to tree-search solutions and human decision-making. *Informatics*, 7:34, 09 2020.

[5] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy P. Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *CoRR*, abs/1712.01815, 2017.

[6] Maciej Swiechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mandziuk. Monte carlo tree search: A review of recent modifications and applications. *CoRR*, abs/2103.04931, 2021.

[7] Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.

[8] Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, March 1995.